

3. To view program execution in memory of the computer using GDB compiler

Theory:

This lab focuses on the compilation of the program using gdb compiler and helps in understand the core concept of Computer architecture and also the address indexing in the system this experiment helps us in understanding the following points

Thus to accomplish this we will implement the following programs, or data structures- stack, queue,

- To implement stack we use an array or linked list and create a program to do push(), pop() and peek() functions.
- To implement a queue we use an array and create a program to do enqueue(), and dequeue() functions.
- To implement linked list we use dynamic allocation using malloc and perform insertion deletion and traversal functions

Implementation:

```
#include <stdio.h>
#define SIZE 5
int stack[SIZE];
int top = -1;
void push(int value) {
    if (top == SIZE - 1) {
        printf("Overflow\n");
        return;
    }
    stack[++top] = value;
}
void pop() {
    if (top == -1) {
        printf("Underflow\n");
        return;
    }
    top--;
}
int main() {
    push(10);
    push(20);
    pop();
    return 0;
}
```

Stack Code

```
#include <stdio.h>
#define SIZE 5

int queue[SIZE];
int front = -1, rear = -1;

void enqueue(int value) {
    if (rear == SIZE - 1) {
        printf("Overflow\n");
        return;
    }
    if (front == -1) front = 0;
    queue[++rear] = value;
}

void dequeue() {
    if (front == -1 || front > rear) {
        printf("Underflow\n");
        return;
    }
    front++;
}

int main() {
    enqueue(1);
    enqueue(2);
    dequeue();
    return 0;
}
```

Queue Code

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* head = NULL;

void insert(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}

void display() {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main() {
    insert(10);
    insert(20);
    display();
    return 0;
}
```

Linked list code

```

Breakpoint 2, push (value=10) at stackds.c:8
8      if (top == SIZE - 1) {
(gdb) n
12      top++;
(gdb) n

Breakpoint 1, push (value=10) at stackds.c:13
13      stack[top] = value; // <-- Put breakpoint here
(gdb) n
14      }
(gdb) n
main () at stackds.c:26
26      (20);
(gdb) n

Breakpoint 2, push (value=20) at stackds.c:8
8      if (top == SIZE - 1) {
(gdb) n
12      top++;
(gdb) n

Breakpoint 1, push (value=20) at stackds.c:13
13      stack[top] = value; // <-- Put breakpoint here
(gdb) n
14      }
(gdb) p top
$1 = 1
(gdb) p stack
$2 = {10, 20, 0, 0, 0}
(gdb) p stack[0]
$3 = 10
(gdb) p stack[1]
$4 = 20
(gdb) print &stack
$5 = (int (*)[5]) 0x55555558030 <stack>
(gdb) p &stack[1]
$6 = (int *) 0x55555558034 <stack+4>
(gdb) p &stack[0]
$7 = (int *) 0x55555558030 <stack>
(gdb) q

```

Stack Output

```

16      front = 0;
(gdb) n
18      rear++;
(gdb) n
19      queue[rear] = value; // <-- Breakpoint can be set here
(gdb) n
20      ("Enqueued: %d\n", value);
(gdb) n
Enqueued: 10
21      }
(gdb) n
main () at queue.c:51
51      (20); // 2nd value
(gdb) n

Breakpoint 1, enqueue (value=20) at queue.c:10
10      if (rear == SIZE - 1) {
(gdb) n
15      if (front == -1)
18      rear++;
(gdb) n
19      queue[rear] = value; // <-- Breakpoint can be set here
(gdb) n
20      ("Enqueued: %d\n", value);
(gdb) n
Enqueued: 20
21      }
(gdb) n
main () at queue.c:52
52      (30); // 3rd value
(gdb) n

Breakpoint 1, enqueue (value=30) at queue.c:10
10      if (rear == SIZE - 1) {
(gdb) n
15      if (front == -1)
18      rear++;
(gdb) n
19      queue[rear] = value; // <-- Breakpoint can be set here
(gdb) n
20      ("Enqueued: %d\n", value);
(gdb) n

```

Queue output

Conclusion:

Successfully compiled and debugged C programs using the GDB. Learned and applied basic debugging commands. Gained practical understanding of Memory layouts ,Pointer operations function call stacks